

□ Distributed Text Analysis in the Cloud – Final Project Report & README

Authors

Name	ID
Ahmad Dahleh	212930820
Ebrahim Taha	213567233

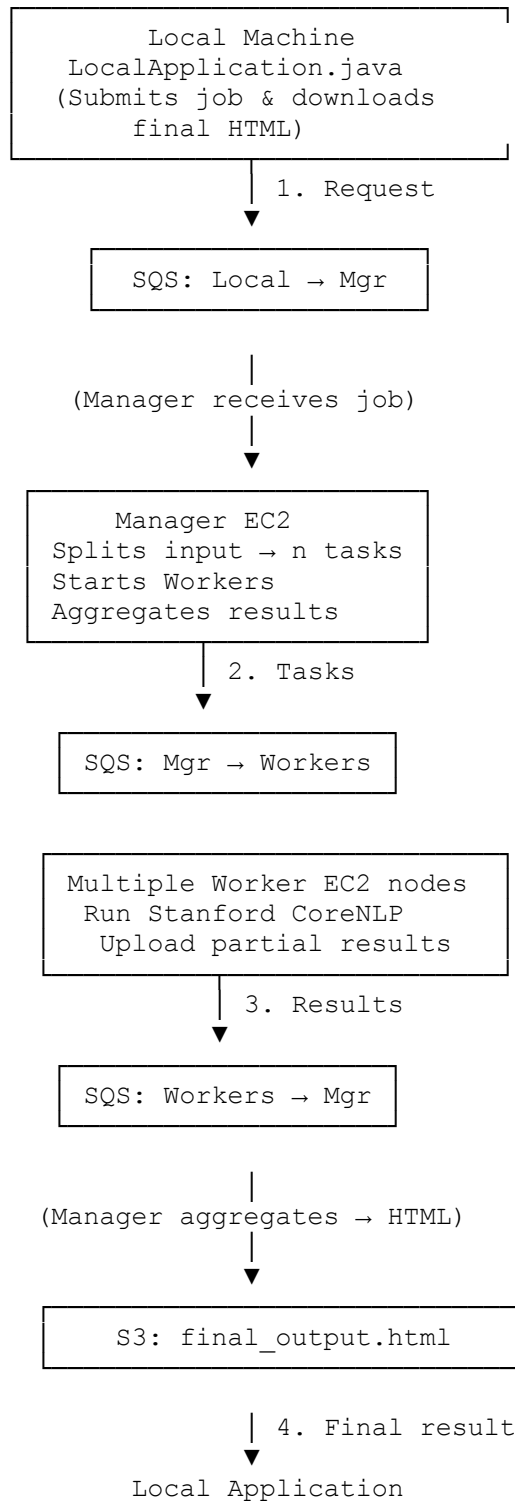
Project Overview

This project implements a **distributed, fault-tolerant text-analysis pipeline** deployed on AWS. The system uses a classic **Scatter–Gather architecture**, where:

- A **Local Application** submits jobs and receives results
- A **Manager EC2 instance** coordinates work
- Multiple **Worker EC2 instances** process text chunks using Stanford CoreNLP
- **S3** provides durable storage
- **SQS** provides asynchronous communication

The system is fully decoupled, asynchronous, and supports multiple simultaneous clients.

High-Level Architecture



Project Structure

```
project-root/
├── src/main/java/dspl/
│   ├── LocalApplication/LocalApplication.java
│   ├── Manager/Manager.java
│   ├── Worker/Worker.java
│   ├── Worker/StanfordAnalyzer.java
│   └── AWS.java
├── src/main/resources/
│   ├── input-sample.txt
│   ├── smallinput.txt
│   └── test.txt
├── pom.xml
└── README.md
```

System Components (Detailed)

1. Local Application

Runs on the user's computer.

Responsibilities:

- Upload input file to S3
- Send job request message to Manager
- Wait for final result
- Download resulting HTML
- Optionally: send a **terminate** command

Uses **UUID** per job → supports multiple simultaneous clients.

2. Manager EC2

The orchestrator of the system.

Performs:

- Queue creation & URL resolution
- Splitting input into n chunks
- Starting Worker EC2 instances
- Sending tasks via SQS
- Tracking task completion via atomic counters
- Aggregating partial results into final HTML
- Uploading final HTML to S3
- Handling system-wide termination

Uses:

- **Fixed Thread Pool** → handles concurrency efficiently
- **ConcurrentHashMap** → stores job states safely

❑ 3. Worker EC2 Instances

Each Worker:

- Receives a task message
- Downloads a text chunk from S3
- Runs **Stanford CoreNLP**
- Generates partial result
- Uploads to S3
- Sends completion message to Manager

Workers are **single-threaded by design** due to CoreNLP's resource usage.

📄 Message Formats (SQS JSON)

Local → Manager

```
{
  "type": "jobRequest",
  "bucket": "dsp-bucket-123",
  "s3Key": "input-sample.txtUUID",
  "taskId": "UUID",
  "n": 3
}
```

Manager → Worker

```
{
  "type": "workerTask",
  "taskId": "UUID",
  "chunkIndex": 0,
  "s3Key": "chunk-0.txt",
  "analysis": "DEPENDENCY"
}
```

Worker → Manager

```
{
  "type": "workerResult",
  "taskId": "UUID",
  "chunkIndex": 0,
  "s3Key": "result-0.html"
}
```

Manager → Local

```
{
  "type": "finalSummary",
  "taskId": "UUID",
  "outputS3Key": "summary-UUID.html"
}
```

Security & AWS Usage

- Uses **IAM roles** (no hard-coded credentials)
 - Temporary session tokens in LocalApp (lab)
 - All communication over **HTTPS**
 - Least-privilege IAM permissions:
 - S3: GetObject/PutObject/ListBucket
 - SQS: SendMessage/ReceiveMessage/DeleteMessage
 - EC2: RunInstances/TerminateInstances
-

Scalability & Load Behavior

- Completely asynchronous
 - Manager scales workers horizontally
 - SQS buffers load spikes
 - System can survive Worker crashes
 - Multiple LocalApps can submit at once
-

Fault Tolerance

Worker crash / freeze

Handled automatically:

1. Worker receives message
2. If Worker dies before deleting it → SQS visibility timeout expires
3. Message becomes visible
4. Another Worker reprocesses the task

No special logic needed — SQS ensures eventual completion.

Manager crash

State is preserved in:

- S3 (uploaded partial results)
 - SQS (pending tasks)
-

Termination Workflow

If LocalApp is run with "terminate":

1. LocalApp sends {"type": "terminate"}
2. Manager finishes current job
3. Uploads final HTML
4. Terminates all Worker EC2 instances
5. Terminates itself

Ensures **graceful, clean shutdown**.

Runtime Measurement

- JAR deployed at: **17:21:56**
- Final HTML uploaded at: **17:40:56**
- **Total runtime: ~19 minutes**

Workers launched: 3 (n=3)

How to Run the System

✓ Run using Maven (recommended)

```
mvn -q exec:java -
Dexec.mainClass="dsp1.LocalApplication.LocalApplication" -
Dexec.args='input-sample.txt output.html 3'
```

Arguments:

1. `input-sample.txt` → input file
 2. `output.html` → output file name
 3. `3` → number of chunks / tasks
 4. Optional: `"terminate"` → shut down entire system
-

Features Summary

Feature	Status
Scatter-Gather Architecture	✓ Implemented
Asynchronous Messaging (SQS)	✓
Durable Storage (S3)	✓
Worker Autoscaling	✓ (manual via Manager)
Multi-Client Support	✓
Fault Tolerance	✓
Graceful Termination	✓
Secure IAM Deployment	✓

Course Information

Course: Distributed Systems Programming – DSP

Project: Distributed Text Analysis in the Cloud

Submission: Final version with full architecture and implementation